BODEKO

Validaciones del backend y guía para el equipo

Revisión técnica | 11 de noviembre de 2026

Documento consolidado

1. Cambios, funcionamiento y despliegue.
2. Lista completa de archivos modificados.
3. Ejemplos de solicitudes HTTP.

Estado para subir a la rama

Los cambios pueden subirse a una rama para revisión. La comprobación del diff terminó correctamente y el escaneo de 406 archivos no detectó secretos. No se realizaron commits ni publicaciones.

Resultados: 226 comprobaciones HTTP, 6 pruebas Java y 18 pruebas del frontend aprobadas, además de la compilación de producción. Las pruebas usaron PostgreSQL temporal y una pasarela simulada; no la base del equipo ni cobros reales.

Antes de desplegar

Ejecutar la prevalidación de duplicados, resolver los datos existentes que entren en conflicto y aplicar las migraciones V6-V8 con Flyway. Coordinar el nuevo contrato de porcentajes e Idempotency-Key con todos los clientes.

Proyecto revisado

```
F:\UTP2026-2\Desarrollo Full Stack\Proyecto 2\bodeko
```

Las rutas del inventario son relativas a esta carpeta. No contiene claves reales, contraseñas de entorno ni tokens de sesión.

1. Cambios, funcionamiento y despliegue

Esta revisión parte de la carpeta `bodeko` compartida con el equipo. No publica ni crea commits. No modifica las migraciones V1-V5 ni utiliza la base de trabajo del equipo para las pruebas.

Cambios solicitados

1. **Tiendas:** RUC y subdominio continúan siendo únicos globalmente. Se añade nombre único por propietario, ignorando mayúsculas y espacios sobrantes. Propietarios distintos pueden tener el mismo nombre comercial; el RUC y el subdominio no se pueden repetir.

2. **Correo:** normalización con Locale.ROOT y un índice único sobre lower(trim(email)), que protege también escrituras simultáneas o registros antiguos con diferencias de mayúsculas.

3. **Productos:** nombre único dentro de cada tienda, ignorando mayúsculas y espacios sobrantes. Incluye borradores, activos, inactivos y archivados; se debe editar o reactivar el registro existente. Otra tienda puede usar el mismo nombre. Las variantes deben tener un nombre distinguible, porque el modelo no dispone de SKU de variante.

4. **Desactivar:** nuevo estado `INACTIVE`. El producto permanece en administración e historial, pero no aparece en el catálogo público ni puede agregarse al carrito o confirmarse en un checkout. Se puede reactivar. Las órdenes ya creadas conservan sus datos históricos.

5. **Descuento:** crear ofertas requiere `discountPercentage`, entre 0.01 y 99.99, con hasta dos decimales. El servidor calcula el importe sobre el precio base vigente, redondea HALF_UP a dos decimales y conserva como mínimo 0.01 de unidad monetaria. `offerPrice` sigue en la respuesta para los consumidores de lectura; ya no es la entrada para una oferta nueva.

Los duplicados se comprueban en servicios para dar un error claro y se refuerzan con índices únicos para cubrir la carrera entre comprobar e insertar. PostgreSQL es la última defensa cuando dos solicitudes llegan juntas. Una migración no elimina, fusiona ni renombra datos existentes automáticamente.

Contrato de ofertas

`POST /api/stores/{storeId}/products/{productId}/offers` con JWT de propietario:

```
{
  "discountPercentage": 25.0,
  "startsAt": "2026-12-01T09:00:00-05:00",
  "endsAt": "2026-12-15T18:00:00-05:00"
}
```

Un producto de 20.00 queda en 15.00; si su precio base cambia a 40.00, queda en 30.00 mientras la oferta siga vigente. El porcentaje no se calcula en el navegador. El rango final debe ser posterior al inicial y no debe superponerse con otra oferta del mismo producto. Se bloquea el producto durante la creación de ofertas para serializar solicitudes concurrentes.

V7 conserva `offer_price` y convierte las ofertas anteriores solo cuando un porcentaje de dos decimales reproduce exactamente su importe. Las demás conservan `discountPercentage: null` y su importe anterior; la interfaz las identifica como ofertas anteriores. No cambia precios históricos de órdenes. Los nuevos datos demo ya incluyen porcentaje cuando es representable.

Cabecera obligatoria en operaciones críticas

Las siguientes rutas requieren `Idempotency-Key`:

- POST `/api/customer/stores/{storeId}/orders`.

- POST `/api/customer/stores/{storeId}/orders/{orderId}/payments`.

- POST `/api/customer/orders/{orderId}/cancel`.

- PATCH `/api/stores/{storeId}/orders/{orderId}/status`.

Usar un UUID por intención de operación. La clave admite de 8 a 128 caracteres alfanuméricos, punto, guion y dos puntos. **Un reintento de red conserva la misma clave y los mismos datos. Una compra o intento nuevo usa otra clave.**

Cambiar los datos con una clave ya usada produce 409 `IDEMPOTENCY_CONFLICT`. Los ejemplos están en `docs/API_VALIDACIONES.http`.

La respuesta exitosa se guarda con los efectos locales en la misma transacción. Un reintento devuelve el resultado original y el mismo estado HTTP, sin repetir checkout, cobro, devolución o transición. Los errores de negocio ya registrados se reproducen con la misma clave; para corregir la operación se necesita un intento nuevo. La autorización y la propiedad de la orden se comprueban antes de reproducir sus datos.

El frontend incluido genera y conserva claves opacas para reintentos. La identidad del checkout también incorpora el id del carrito: una compra futura con los mismos datos fiscales recibe una clave distinta. No se guarda el token de pago en sessionStorage: solo una huella y la clave. Los otros clientes del equipo deben enviar esta cabecera por su cuenta.

Cómo se evita una venta doble

El carrito no reserva unidades al agregar productos. La reserva se efectúa al crear la orden, como en el diseño previo. Dos clientes pueden tener la última unidad en sus carritos; al confirmar, solo uno obtiene la orden y el otro recibe 409 por stock insuficiente.

Se conservan los bloqueos de carrito e inventario existentes y se refuerzan las escrituras concurrentes:

- La creación/modificación del carrito bloquea su fila; un cerrojo transaccional por cliente y tienda evita crear dos carritos abiertos o perder incrementos simultáneos.

- Checkout bloquea productos por id ordenado y refresca sus datos después de esperar el cerrojo. Así no usa un estado o precio anterior a una modificación concurrente.

- El inventario se consulta con bloqueo antes de comprobar y descontar. La actualización administrativa también usa ese bloqueo.

- Las órdenes siguen bloqueándose antes de pagar o cancelar y su estado se refresca dentro del bloqueo para descartar la versión cargada por una lectura previa. El índice parcial de pagos evita más de un pago APPROVED por orden.

- La creación de órdenes y su resultado idempotente se confirman juntos. Los bloqueos se toman en un orden consistente para reducir interbloqueos. Los conflictos de concurrencia esperados se devuelven como 409.

Resultado incierto del proveedor

La tabla `api_idempotency` conserva IN_PROGRESS, COMPLETED, FAILED o UNCERTAIN. Antes de llamar al proveedor, se confirma por separado el marcador `external_started`. El cuerpo recibido no se guarda; su huella SHA-256 permite comparar reintentos. Las respuestas guardadas son datos privados del servidor y no se deben exportar con datos reales.

Si el proveedor pudo haber cobrado o devuelto dinero y se pierde su respuesta, no existe una transacción distribuida que lo deshaga. El marcador queda UNCERTAIN y bloquea nuevas operaciones externas sobre esa orden, incluso con otra clave o desde administración. Una caída del proceso puede dejar IN_PROGRESS y también requiere revisión antes de repetir un movimiento externo.

No borrar estos registros ni cambiar claves para saltar el bloqueo. El equipo debe conciliar la orden con el proveedor usando su número y metadatos, y completar o anular el movimiento mediante un procedimiento auditado. No se agregó un endpoint de conciliación ni se implementaron webhooks de Culqi. No se afirma idempotencia nativa del proveedor: esta protección está en la aplicación. No hay reintentos automáticos de cobros inciertos.

Otras entradas protegidas

- Contraseñas de hasta 72 bytes UTF-8, también en login, para no disparar una excepción de BCrypt con entradas largas.

- Nombres y cadenas de las solicitudes sin el carácter nulo, que PostgreSQL no admite en texto.

- Precio nuevo de producto entre 0.01 y 100000.00, con dos decimales; descripción de hasta 10000 caracteres.

- URL de imagen opcional: http/https con host y sin credenciales.

- Hasta 10000 unidades por producto en carrito, incluyendo incrementos acumulados, y hasta 1000000 en los campos administrativos de inventario.

- El total de checkout y los importes enviados a la pasarela deben caber en un entero positivo de céntimos. Un exceso devuelve un error controlado.

- Las cantidades fraccionarias y los estados enviados como números se rechazan: no se truncan ni se convierten silenciosamente. Errores por UUID, enum, parámetros ausentes, JSON inválido y tipo de contenido incorrecto tienen respuestas 4xx. Los mensajes generales y de campos son legibles en español. Los códigos previos se mantienen.

- Los conflictos de integridad registran el tipo del conflicto, sin escribir el SQL ni el valor de un correo o documento en el log de ese manejador.

Migraciones antes del despliegue

1. Ejecutar **solo lectura** `scripts/db/03_prevalidar_mejoras.sql` en la base que se actualizará. Revisar las filas devueltas con el equipo; el resultado puede contener datos privados y no debe subirse al repositorio.

2. Resolver duplicados de correos, nombres y subdominios, y pagos APPROVED múltiples si existieran. No se eliminan automáticamente porque pueden tener órdenes asociadas.

3. Dejar que Flyway aplique V6 (unicidad e INACTIVE), V7 (ofertas) y V8 (idempotencia). V1-V5 se conservaron.

4. Desplegar conjuntamente el contrato de ofertas y las cabeceras de operaciones críticas en todos los clientes.

Si existen duplicados, V6 se detiene en lugar de perder datos o desplegar una protección incompleta. No se cambiaron pom.xml, credenciales ni servicios cloud. Las pruebas se ejecutaron con Java 21 usando `-Djava.version=21`; el proyecto conserva Java 23 declarado.

Respuestas que los clientes deben manejar

Estado / código	Significado
400 VALIDATION_ERROR, INVALID_REQUEST, IDEMPOTENCY_REQUIRED	Revisar campos, tipos o clave ausente.
401 / 403	Sesión o permisos.
404	Recurso inexistente o no accesible por esa cuenta.
409 PRODUCT_DUPLICATE, STORE_DUPLICATE, USER_EMAIL_EXIST	Duplicado controlado.
409 INVENTORY_INSUFFICIENT_STOCK	Otra compra consumió las unidades; actualizar el carrito.
409 IDEMPOTENCY_CONFLICT	No reutilizar una clave con datos diferentes.
409 OPERATION_IN_PROGRESS	Conservar clave y consultar/reintentar después de que termine.
409 OPERATION_UNCERTAIN	Revisar con el proveedor; no repetir el movimiento.
402 PAYMENT_DECLINED	Rechazo de pago conocido y persistido.
422 PAYMENT_REFUND_REJECTED	Devolución rechazada; los cambios locales se revierten.
502 PAYMENT_GATEWAY_UNAVAILABLE	No se confirmó una respuesta de la pasarela. Consultar estado; puede requerir conciliación.

Una respuesta 4xx ante una entrada inválida o un conflicto es comportamiento correcto. No es posible prometer que una caída de PostgreSQL o del proveedor nunca produzca un error. Las pruebas buscan evitar 500 inesperados en los escenarios verificados, sobreventa y efectos duplicados.

Verificación y pruebas

- 115 comprobaciones HTTP nuevas: duplicados concurrentes, INACTIVE, porcentajes, entrada inválida, última unidad, carrito concurrente, replay, conflicto de huellas, cobro/devolución únicos y resultado incierto.

- Regresión: 55 comprobaciones del flujo inicial y 56 de órdenes del cliente, ambas adaptadas al contrato nuevo.

- 18 pruebas del frontend y compilación de producción.

- Pruebas Java: contexto de Spring, reglas monetarias y actualización V5 a V8 con ofertas antiguas en PostgreSQL temporal. Consultar los reportes JUnit al ejecutarlas.

Nueva batería: `scripts/qa/verificar-validaciones.mjs --disposable`. Exige QA_API_URL, QA_MOCK_URL, una API local conectada a base **vacía desechable**, y `python scripts/qa/culqi-pruebas.py --disposable` en otra terminal. El simulador de QA escucha solo en 127.0.0.1:8210 y permite el caso de respuesta perdida; no es el simulador de uso cotidiano ni una pasarela real.

Para la prueba Java de actualización: activar BODEKO_UPGRADE_TEST=true y BODEKO_UPGRADE_DB=jdbc:postgresql://127.0.0.1:55433/bodeko_upgrade_qa, con esa base vacía y desechable. Sin la variable de activación, esa prueba se omite para no crear ni migrar una base externa por accidente. No ejecutar las pruebas que crean datos contra una base de trabajo.

Los archivos cambiados se enumeran en `docs/ARCHIVOS_VALIDACIONES.md`. No se crearon commits ni se publicó el repositorio.

Referencia de configuración JSON: las propiedades de EnumFeature en Jackson 3 usan `spring.jackson.datatype.enum.*`, según la documentación oficial de Spring Boot.

2. Lista de archivos modificados

Inventario de 78 archivos. Incluye backend, migraciones, pruebas, documentación y las adaptaciones del frontend al nuevo contrato. La columna de tipo indica si el archivo se añadió o modificó.

Archivo	Tipo
docs/API_VALIDACIONES.http	nuevo
docs/ARCHIVOS_VALIDACIONES.md	nuevo
docs/VALIDACIONES_BACKEND.md	nuevo
frontend/README.md	modificado
frontend/docs/CAMBIOS_BACKEND.md	modificado
frontend/docs/CODIGO_COMPLETO.md	modificado
frontend/src/config/contracts.test.ts	modificado
frontend/src/config/idempotency.test.ts	nuevo
frontend/src/config/idempotency.ts	nuevo
frontend/src/features/checkout/hooks/useCheckout.ts	modificado
frontend/src/features/checkout/services/checkoutService.ts	modificado
frontend/src/features/customer-orders/services/customerOrderService.test.ts	modificado
frontend/src/features/customer-orders/services/customerOrderService.ts	modificado
frontend/src/features/offers/OffersPage.tsx	modificado
frontend/src/features/offers/components/OfferForm.tsx	modificado
frontend/src/features/offers/types/index.ts	modificado
frontend/src/features/orders/services/orderService.ts	modificado
frontend/src/features/payments/services/paymentService.ts	modificado
frontend/src/features/products/ProductsPage.tsx	modificado
frontend/src/features/products/components/ProductForm.tsx	modificado
frontend/src/features/products/types/index.ts	modificado
frontend/src/utils/errors.ts	modificado
frontend/src/utils/format.ts	modificado
frontend/tools/verify-api.mjs	modificado
frontend/tools/verify-customer-orders.mjs	modificado
scripts/db/01_datos_demo.sql	modificado
scripts/db/03_prevalidar_mejoras.sql	nuevo
scripts/db/README.md	modificado
scripts/qa/culqi-pruebas.py	nuevo

Archivo	Tipo
scripts/qa/verificar-validaciones.mjs	nuevo
src/main/java/com/reydiazz/bodeko/core/category/web/request/CreateCategoryRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/category/web/request/EditCategoryRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/inventory/service/InventoryServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/inventory/web/request/EditLowStockThresholdRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/inventory/web/request/EditStockQuantityRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/order/core/service/OrderServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/order/core/web/controller/OrderController.java	modificado
src/main/java/com/reydiazz/bodeko/core/order/core/web/request/CheckoutRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/order/customer/service/CustomerOrderServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/order/customer/web/controller/CustomerOrderController.java	modificado
src/main/java/com/reydiazz/bodeko/core/order/management/service/OrderManagementServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/order/management/web/controller/OrderManagementController.java	modificado
src/main/java/com/reydiazz/bodeko/core/payment/gateway/culqi/CulqiPaymentGateway.java	modificado
src/main/java/com/reydiazz/bodeko/core/payment/service/PaymentServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/payment/web/controller/PaymentController.java	modificado
src/main/java/com/reydiazz/bodeko/core/payment/web/request/CreatePaymentRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/core/model/enums/ProductStatus.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/core/repository/ProductRepository.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/core/service/ProductServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/core/web/request/CreateProductRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/core/web/request/EditProductRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/offer/component/ProductOfferMapper.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/offer/model/entity/ProductOffer.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/offer/repository/ProductOfferRepository.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/offer/service/ProductOfferServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/offer/web/request/CreateProductOfferRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/product/offer/web/response/ProductOfferResponse.java	modificado
src/main/java/com/reydiazz/bodeko/core/shopping/cart/service/ShoppingCartServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/shopping/cart/web/request/AddCartItemRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/shopping/cart/web/request/EditCartItemQuantityRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/store/repository/StoreRepository.java	modificado

Archivo	Tipo
src/main/java/com/reydiazz/bodeko/core/store/service/StoreServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/core/store/web/request/CreateStoreRequest.java	modificado
src/main/java/com/reydiazz/bodeko/core/user/core/service/UserServiceImpl.java	modificado
src/main/java/com/reydiazz/bodeko/security/auth/web/request/LoginRequest.java	modificado
src/main/java/com/reydiazz/bodeko/security/auth/web/request/RegisterCustomerRequest.java	modificado
src/main/java/com/reydiazz/bodeko/security/auth/web/request/RegisterOwnerRequest.java	modificado
src/main/java/com/reydiazz/bodeko/shared/exception/GlobalExceptionHandler.java	modificado
src/main/java/com/reydiazz/bodeko/shared/idempotency/IdempotencyService.java	nuevo
src/main/java/com/reydiazz/bodeko/shared/idempotency/StoredFailure.java	nuevo
src/main/java/com/reydiazz/bodeko/shared/validation/NormalizedText.java	nuevo
src/main/java/com/reydiazz/bodeko/shared/validation/ValidationError.java	nuevo
src/main/resources/application.properties	modificado
src/main/resources/db/migration/V6__unique_names_and_inactive_products.sql	nuevo
src/main/resources/db/migration/V7__percentage_offers.sql	nuevo
src/main/resources/db/migration/V8__persistent_idempotency.sql	nuevo
src/test/java/com/reydiazz/bodeko/MigrationUpgradeTests.java	nuevo
src/test/java/com/reydiazz/bodeko/PercentageOfferTests.java	nuevo

3. Ejemplos de solicitudes HTTP

Reemplazar los identificadores y el JWT por los del entorno de desarrollo. Usar el JWT del propietario o del cliente según la operación. Las variables con REEMPLAZAR son marcadores, no credenciales reales.

Cada operación nueva usa otro UUID. Un reintento de la misma operación conserva su UUID y su cuerpo. El token tkn_test_ok corresponde al simulador; no representa un token real de Culqi.

Variables de ejemplo

```
# Ejemplos para adaptar con los ids y el JWT del entorno de desarrollo.
# operationKey: usar otro UUID en una intención nueva; conservarlo en un reintento de red.
@baseUrl = http://localhost:8080/api
@token = REEMPLAZAR_JWT_DE_SESION
@storeId = REEMPLAZAR_ID_TIENDA
@productId = REEMPLAZAR_ID_PRODUCTO
@orderId = REEMPLAZAR_ID_ORDEN
@operationKey = 11111111-1111-4111-8111-111111111111
```

Desactivar producto (propietario)

```
PATCH {{baseUrl}}/stores/{{storeId}}/products/{{productId}}/status
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "status": "INACTIVE"
}
```

Oferta porcentual (propietario)

```
POST {{baseUrl}}/stores/{{storeId}}/products/{{productId}}/offers
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "discountPercentage": 25.0,
  "startsAt": "2026-12-01T09:00:00-05:00",
  "endsAt": "2026-12-15T18:00:00-05:00"
}
```

Checkout (cliente)

```
POST {{baseUrl}}/customer/stores/{{storeId}}/orders
Authorization: Bearer {{token}}
Idempotency-Key: {{operationKey}}
Content-Type: application/json

{
  "invoiceType": "RECEIPT",
  "documentType": "DNI",
  "documentNumber": "12345678"
}
```

Pago (cliente): cambiar operationKey para esta operación nueva

```
POST {{baseUrl}}/customer/stores/{{storeId}}/orders/{{orderId}}/payments
Authorization: Bearer {{token}}
Idempotency-Key: {{operationKey}}
Content-Type: application/json

{
  "sourceId": "tkn_test_ok"
}
```

Cancelación propia (cliente): otra operationKey

```
POST {{baseUrl}}/customer/orders/{{orderId}}/cancel
Authorization: Bearer {{token}}
Idempotency-Key: {{operationKey}}
```

Cancelación administrativa (propietario): otra operationKey

```
PATCH {{baseUrl}}/stores/{{storeId}}/orders/{{orderId}}/status
Authorization: Bearer {{token}}
Idempotency-Key: {{operationKey}}
Content-Type: application/json

{
  "status": "CANCELLED"
}
```

Respuestas y pruebas manuales

Una entrada inválida devuelve 400; un duplicado o falta de stock concurrente devuelve 409. Repetir una operación con la misma clave y datos debe conservar su resultado. Reutilizar la clave con otros datos devuelve 409 IDEMPOTENCY_CONFLICT.

Si se pierde la respuesta del proveedor, el resultado puede quedar incierto: requiere conciliación manual. No cambiar la clave ni borrar el registro para forzar otro cobro o devolución.

Documentos de origen

docs/VALIDACIONES_BACKEND.md

docs/ARCHIVOS_VALIDACIONES.md

docs/API_VALIDACIONES.http